

Compiler-Scheduled Sparsity (CSS)

Native 2:4 and 4:8 Structured Sparsity as a First-Class Type with Progressive Training Recipes and CFTP/CFIP/WRGA Composition in NeuralScript

Authors: Brandon Wiemer **Date:** April 2026 **Status:** Research Proposal

Abstract

We propose **CSS (Compiler-Scheduled Sparsity)**, a structured-sparsity technique for NSL that elevates hardware-accelerated sparsity patterns (2:4 on Ampere/Hopper, 4:8 NVFP4 on Blackwell) from a user-instrumented optimization to a first-class compiler feature. NSL's type system already has `Sparse<[shape], dtype, format>` as scaffolding; CSS develops the type into a working technique. Contributions: (1) `Sparse<...>` as a native tensor type with compile-time pattern checking and automatic mixed-sparse/dense matmul dispatch; (2) compile-time synthesis of progressive pruning schedules from a user-declared target sparsity and recovery budget; (3) hardware-aware format selection (2:4 FP16/FP8 on Hopper, 4:8 NVFP4 on Blackwell); (4) composition with CFTP (sparse FP8 weights), CFIP (sparse + low-precision interior formats), WRGA (dense adapters on sparse base weights), and CSHA (sparse attention weight matrices); (5) progressive pruning schedule synthesis that handles accuracy recovery without user tuning. Expected impact: 1.3-1.8× inference speedup on Hopper H100 (matching Red Hat Neural Magic's Sparse-FP8 Llama); 2× theoretical speedup where kernel maturity permits; automatic accuracy recovery via progressive pruning without the user-by-hand iteration that currently limits 2:4 adoption.

1. Structured Sparsity as a Missing First-Class Feature

1.1 What hardware provides

Modern NVIDIA tensor cores accelerate structured sparsity:

Ampere (A100, SM_80):

- **2:4 pattern:** in every group of 4 contiguous values, at least 2 must be zero.
- Theoretical 2× matmul throughput vs dense.
- Sparse tensor cores consume only non-zero values and 4-bit index metadata.

Hopper (H100, SM_90):

- 2:4 pattern extended to FP8 types.
- Sparse FP8 kernels achieve $1.7\times$ lower latency and $1.5\times$ higher throughput vs dense FP8 on Llama 3.1 8B (Red Hat Neural Magic, 2024).
- Combined $4\times$ FP8 compression + $2\times$ sparsity = $2.5\times$ vs BF16 dense.

Blackwell (B200, SM_100):

- **4:8 pair-wise pattern:** in every group of 8, two pairs of 2 consecutive values must be zero. Same 50% sparsity but aligned with 5th-generation tensor core granularity.
- Hardware support for NVFP4 $\times$ 4:8 sparsity.
- Theoretical gains stack with NVFP4's $4\times$ compression.

The hardware is ready. What's missing is adoption.

1.2 Why adoption is limited

Despite the theoretical $2\times$ throughput, production inference (OpenAI, Anthropic, Google) largely does not use 2:4 sparsity. The gap between theoretical and practical:

1. **Accuracy recovery difficulty.** Post-training pruning without fine-tuning loses 1-3 points on standard benchmarks. With sparse fine-tuning (SparseGPT, Wanda) it drops below 1 point, but requires additional training pipeline.
2. **Kernel maturity.** CUTLASS sparse kernels are less optimized than cuBLAS dense kernels. On Hopper, measured speedup is often $1.27\text{-}1.3\times$ instead of the theoretical $2\times$.
3. **TDP-limited workloads.** Many training/inference systems are power-limited rather than compute-limited; the $2\times$ compute doesn't translate to wall-clock.
4. **Operational overhead.** Maintaining a sparse-model pipeline alongside dense doubles deployment complexity.
5. **Uneven benefit across architectures.** Some model architectures (heavily GQA, heavily MoE) don't have the dense-matmul structure where sparsity helps most.

Current production status: Chinese AI labs use 2:4 more actively than Western labs; Meta has experimented in Llama but not rolled out broadly; Red Hat Neural Magic + NVIDIA have a clean Sparse-FP8 Llama reference but it remains demo-grade.

1.3 Why NSL should have this

NSL's type system already declares `Sparse<[shape], dtype, format>`. The scaffolding was placed in anticipation of a future paper. That paper is now.

NSL's unique opportunity: the type system plus compile-time scheduling can address the adoption gaps:

- **Accuracy recovery via progressive pruning schedules** synthesized at compile time rather than tuned by hand.
- **Kernel selection** between CUTLASS sparse and cuBLAS dense based on measured hardware performance, not assumptions.
- **Composition with CFTP/CFIP** unlocks sparse-and-quantized paths that external tools treat as separate.
- **Type-system pattern checking** prevents pattern mismatches at compile time (a common runtime bug in sparse-kernel integration).

Putting structured sparsity into a compiler's type system and decision pipeline makes it a routine choice rather than a research project.

2. Seven Compiler-Scheduled Sparsity Innovations

2.1 Innovation 1: `Sparse<...>` as a first-class type

NSL's type system gains a working `Sparse<...>`:

```
ns!

// 2:4 sparse FP16 weight matrix
weight: Sparse<[4096, 4096], f16, "2:4">

// 4:8 sparse NVFP4 weight matrix (Blackwell)
weight_b: Sparse<[4096, 4096], nvfp4, "4:8">

// Dense weight (standard)
weight_d: Tensor<[4096, 4096], f16>
```

The type carries:

- Shape (as usual).
- Dtype (as usual).
- Pattern ("2:4", "4:8", or future variants).
- Implicit storage: non-zero values + metadata (indices).

Type-level checking:

- Operations between `Sparse<...>` and `Tensor<...>` emit mixed-sparse/dense kernels (CUTLASS).
- Operations between two `Sparse<...>` tensors require *compatible* patterns. Two 2:4 matrices can multiply; a 2:4 and a 4:8 cannot (at the type level; the compiler would need to densify one).
- Shape $\times$ pattern compatibility: the sparse dimension must be a multiple of the pattern's group size. 2:4 sparsity on an inner dimension of 17 is rejected at compile time.

2.2 Innovation 2: Progressive pruning schedule synthesis

User declares intent:

```
nsl

@sparsity(pattern="2:4", recovery_epochs=10, warmup_epochs=2)
weight: Sparse[4096, 4096], f16, "2:4">
```

CSS synthesizes a training-time schedule from these parameters:

- **Epochs 0 to `warmup_epochs`:** train dense. Establish a baseline.
- **Epochs `warmup_epochs` to `warmup_epochs + recovery_epochs/2`:** progressively introduce sparsity. Per-epoch, increase the sparsity ratio linearly: at epoch i , mask the bottom- k % of weights by magnitude where k ramps from 0% to 50%.
- **Epochs `warmup_epochs + recovery_epochs/2` to `warmup_epochs + recovery_epochs`:** fixed 50% sparsity with 2:4 pattern. Fine-tune to recover accuracy.
- **Epochs beyond recovery:** standard training with fixed sparsity mask.

The schedule is exposed as a runtime-visible artifact that the trainer loop consults for each step. No user-level epoch tracking required.

This is the Tencent progressive recipe formalized in the compiler: "Train dense for a bit, progressively add sparsity, recover accuracy." Tencent reports 1.3-1.8 $\times$ end-to-end acceleration using this approach in production (with quantization).

2.3 Innovation 3: Mask recomputation schedule

Within the progressive pruning phase, CSS offers two mask-update policies:

- **Static mask:** compute the 2:4 mask once, freeze it, continue training with the fixed mask. Simple. Lower accuracy recovery.
- **Dynamic mask:** recompute the 2:4 mask every K steps based on current weight magnitudes. Higher accuracy recovery. More compute overhead.

For dynamic masks, the default K is determined by the model size: smaller models recompute more often (K=50), larger models less often (K=500). User can override.

Dynamic mask recomputation is critical for accuracy recovery, matching PyTorch's semi-structured sparsity DINOv2 training observations: dynamically updating the mask during training produces virtually no accuracy degradation (82.8 → 82.7 on ImageNet top-1) vs static mask which often drops 1-2 points.

2.4 Innovation 4: Hardware-format selection

CSS selects the underlying sparse format based on target GPU:

- **A100 (SM_80):** 2:4 FP16 or 2:4 BF16.
- **H100 (SM_90):** 2:4 FP8 (E4M3 for weights, E5M2 for activations) — leverages M35 FP8 MMA.
- **B200 (SM_100):** 4:8 NVFP4.

The `@sparsity(pattern=...)` annotation can be explicit or `pattern="auto"` — the latter lets CSS pick based on `--gpu-arch`. Cross-hardware deployment uses NSL's multi-target compilation to emit different sparsity paths per architecture.

Kernel selection decision logic:

```
if target == Blackwell and dtype == NVFP4:
    use 4:8 sparse NVFP4 tensor cores
elif target == Hopper and dtype in {FP8_E4M3, FP8_E5M2}:
    use 2:4 sparse FP8 via CUTLASS
elif target == Ampere+ and dtype in {FP16, BF16}:
    use 2:4 sparse via CUTLASS
else:
    fallback to dense (with warning)
```

2.5 Innovation 5: Composition with CFTP (Sparse + FP8 + 4-bit residual)

CFTP's FP8+4-bit residual decomposes each weight tile into a high-precision part (FP8) and a low-precision residual (4-bit). CSS composes with this:

```
W_sparse_cftp = Sparse<[M, K], CFTP, "2:4">

# Decomposes to:
W_hi_sparse: Sparse<[M, K], FP8_E4M3, "2:4"> # sparse FP8 primary
W_lo_sparse: Sparse<[M, K], INT4, "2:4">      # sparse 4-bit residual
```

Both components share the same 2:4 mask (the sparsity pattern is applied to the combined weight, then decomposed). The matmul path:

```
out = sparse_fp8_matmul(x, W_hi_sparse) + * sparse_int4_matmul(x, W_lo_sparse)
```

The CUTLASS sparse FP8 path handles the primary matmul; the INT4 residual uses a second sparse path. Combined throughput is approximately $2\times$ the dense CFTP path (from the 2:4 sparsity).

This is the composition that Red Hat Neural Magic's Sparse-FP8 Llama demonstrates end-to-end, at $1.7\times$ latency / $1.5\times$ throughput on Hopper H100 for Llama 3.1 8B. CSS aims for parity with that result on NSL.

2.6 Innovation 6: Composition with CFIP interior formats

A 2:4 sparse weight's non-zero values can still be CFIP-format-assigned. Sparsity operates at the structural level (where are the zeros?); CFIP operates at the precision level (how many bits for each non-zero?). The two are orthogonal:

```
# Sparse weights, CFIP-interior-format for the non-zeros
W_sparse_il2 = Sparse<[M, K], IL2, "2:4">
```

Non-zero values stored at 2-bit per value; zero mask handled by the 2:4 metadata. Combined compression vs dense FP16: $50\% (\text{sparsity}) \times 8 \times (\text{IL2 2-bit}) = 16\times$ density reduction.

This is a particularly aggressive corner of the design space. Accuracy may suffer; CSS offers it but defaults to more conservative combinations (2:4 with FP8 or FP16) unless the user explicitly opts in.

2.7 Innovation 7: Composition with WRGA (dense adapters on sparse base)

Low-rank adapters don't naturally admit 2:4 sparsity — the rank- r adapter matrix has no 4-element grouping along the relevant dimension when r is small ($r = 8$ or 16). CSS handles this cleanly:

```
base: Sparse<[d_model, d_model], f16, "2:4">
adapter_a: Tensor<[d_model, rank], f16>      # dense
adapter_b: Tensor<[rank, d_model], f16>      # dense
```

The forward pass is a sparse base MMA + a dense low-rank MMA:

```
out = sparse_matmul(x, base) + (x @ adapter_a) @ adapter_b
```

WRGA's existing fused LoRA kernel is extended: the base MMA uses the sparse tensor core path, the adapter MMA uses the dense path, both accumulate into the same output register. Latency-wise, this is very close to pure sparse (the adapter MMA is small).

This lets fine-tuning proceed on a sparse base model, which is the dominant deployment pattern (large base model pre-trained with sparsity; small adapter fine-tunes per task).

3. Implementation Architecture

3.1 Pipeline position

CSS operates across stages 2, 3, 4, 5, 6, and 7.

Stage 2 (semantic): `Sparse<...>` types resolved; pattern compatibility checked.

Stage 3 (annotation resolution): `@sparsity(pattern=..., schedule=...)` annotations processed. Mask tensors allocated.

Stage 4 (WGGO): per-layer decisions on whether to use the sparse path (based on hardware, layer shape, accuracy budget). Pruning schedule synthesized.

Stage 5 (MIR lowering): sparse MMA ops replace dense MMA ops where applicable; mask tensors wired into the kernel inputs.

Stage 6 (codegen): CUTLASS sparse kernel calls emitted for A100/H100; NVFP4 sparse kernels emitted for Blackwell; dense fallback kernels emitted for unsupported hardware.

Stage 7 (runtime): pruning schedule drives per-step mask updates during training. The inference path uses fixed masks established at end of training.

3.2 Mask storage

Sparse tensor storage layout:

```
struct SparseTensor<T, "2:4"> {  
    // Non-zero values: half the elements of a dense tensor  
    values: [T; M * K / 2],  
  
    // 4-bit index metadata: 2 bits per group of 4 indicating which 2 of 4 are non-zero  
    // 16 groups of 4 per 8 bytes; so M*K/16 bytes for indices  
    indices: [u4; M * K / 2],  
}
```

For Blackwell's 4:8 pattern:

```
struct SparseTensor<T, "4:8"> {  
    // Non-zero values: half of elements  
    values: [T; M * K / 2],  
  
    // 3-bit index metadata per group of 8 (which of 4 pair positions are non-zero)  
    indices: [u3; M * K / 8],  
}
```

3.3 CUTLASS integration

For 2:4 sparse kernels, CSS emits calls into CUTLASS's `cutlass::gemm::device::SparseGemm` template. The template is instantiated per `(dtype_a, dtype_b, dtype_accum, shape, layout)` tuple; CSS emits one instantiation per unique tuple the model requires.

For NVFP4 sparse kernels, CSS uses CUTLASS 3.6+ which added NVFP4 + 4:8 sparsity support. The template parameters include the NVFP4 scale-factor type (FP8 E4M3).

The CUTLASS library is linked as a static library in the NSL binary; the emitted kernel calls are direct function pointers into CUTLASS's compiled templates.

3.4 Pruning schedule execution

The pruning schedule is a runtime artifact executed by the trainer loop. For a dynamic mask (recompute every K steps):

```
// At step T:
if T % K == 0 and T < schedule_end:
    for each @sparsity-annotated tensor:
        W = load tensor
        target_density = schedule.density_at(T)
        mask = compute_2_4_mask(W, target_density)
        apply_mask(W, mask)
        store mask for subsequent kernel launches
```

The mask computation is a single kernel: for each group of 4 elements, keep the top-2 by absolute magnitude, zero the rest. This is a fast kernel (a few microseconds per tensor on H100).

For static masks, this runs once at schedule-end. For dynamic masks, it runs periodically.

3.5 Fine-tune recovery

During the recovery phase of training, the mask is either frozen (static) or periodically recomputed (dynamic). The weights themselves continue to be updated by the optimizer; zeroed positions remain zero because the gradient-descent update preserves the mask (the update is masked before being applied).

PyTorch's semi-structured sparsity paper reports that dynamically recomputed masks during training produce virtually no accuracy degradation on DINOv2 ViT-L (82.8 → 82.7 on ImageNet top-1). CSS expects similar results.

4. Integration with NSL Pipeline

4.1 New invariants

I-42 — `Sparse<...>` shapes respect pattern alignment. Inner dimension must be a multiple of the pattern's group size (4 for 2:4, 8 for 4:8). Enforced at type-check time.

I-43 — Mask recomputation is atomic with respect to kernel launches. Mask update and subsequent matmul are ordered; no kernel uses a half-updated mask.

I-44 — Pruning schedule is monotonic in sparsity. Sparsity never decreases across epochs within the schedule. The schedule only transitions "denser $\rightarrow$ sparser $\rightarrow$ fixed."

I-45 — Sparse and dense kernels produce numerically-comparable outputs. The sparse matmul is a mathematically-different operation (zeros are multiplied out), but the accuracy difference vs dense is bounded by the pruning magnitude.

I-46 — CSS composes with CFTP at the tile level. The 2:4 pattern is enforced on the combined CFTP weight ($W_{hi} + s \cdot W_{lo}$), not independently on W_{hi} and W_{lo} .

4.2 Failure modes

F-28 — Pattern mismatch at type level. Symptom: compile error about incompatible `Sparse<...>` types. Root cause: matmul of 2:4 and 4:8 sparse without explicit densification. Mitigation: clear type error with suggestion (densify one operand, or use matching patterns).

F-29 — Mask update stale. Symptom: output values differ from expected after a mask recompute. Root cause: stale mask used by a kernel launched during mask update. Mitigation: runtime synchronization (sync before mask-update kernel, sync after).

F-30 — Accuracy recovery failure. Symptom: after progressive pruning, loss is much higher than dense baseline. Root cause: pruning schedule too aggressive, recovery epochs too few. Mitigation: CSS emits a warning if the training-loss slope during recovery phase is unusually steep; user can extend `recovery_epochs`.

F-31 — NVFP4 sparse on Hopper. Symptom: compile-time error about unsupported format+sparsity combination. Root cause: NVFP4 4:8 requires Blackwell. Mitigation: CSS checks `--gpu-arch` before allowing NVFP4 sparse; falls back to FP8 2:4 on Hopper with a warning.

F-32 — Sparse base + adapter fine-tuning instability. Symptom: fine-tuning loss diverges. Root cause: the sparse base's 2:4 pattern interacts poorly with the adapter's gradient flow. Mitigation: WRGA's fused LoRA path is sparse-base-aware; pure dense adapter is the stable default.

5. Evaluation Plan

5.1 Baselines

1. **Dense FP16**. Accuracy/speed baseline.
2. **Dense FP8 (CFTP)**. Current best NSL baseline.
3. **2:4 sparse FP16 (PyTorch semi-structured)**. Reference implementation.
4. **2:4 sparse FP8 (Red Hat Neural Magic)**. Production reference.
5. **CSS 2:4 FP16**.
6. **CSS 2:4 FP8 (with CFTP)**.
7. **CSS 4:8 NVFP4 (Blackwell only)**.

5.2 Benchmarks

- **Inference throughput** on Llama-style 8B, Llama-style 70B, NSLCoder-50M.
- **Fine-tuning throughput** with WRGA adapters on a sparse base.
- **Accuracy** on MMLU, HumanEval, wikitext-2 perplexity.
- **Pruning schedule sensitivity** — ablate recovery_epochs and warmup_epochs.

5.3 Specific targets

Configuration	Target speedup	Accuracy target
Dense FP16 → CSS 2:4 FP16 on H100	1.3×	Within 0.5 MMLU points
Dense FP8 → CSS 2:4 FP8 on H100	1.7×	Within 1 MMLU point
Dense NVFP4 → CSS 4:8 NVFP4 on B200	1.5×	Within 0.5 MMLU points
Fine-tuning on sparse base	Equal	Equal accuracy

5.4 Correctness tests

1. **Type-check failures** for invalid pattern compositions.
2. **2:4 mask validity** — every group of 4 values in the stored tensor has exactly 2 non-zeros.
3. **Mask recomputation stability** — dynamic-mask training produces stable loss curves (no sudden spikes at mask-update boundaries).
4. **Sparse + adapter composition** — fine-tuning loss on sparse base matches fine-tuning loss on dense base within tolerance.
5. **Hardware format correctness** — NVFP4 4:8 on Blackwell produces the same logical result as 2:4 FP16 on Ampere/Hopper (within precision).

5.5 Implementation phases

Phase	Scope	Sessions
1	<code>Sparse<...></code> type system extensions	2
2	Pattern compatibility checking	1
3	Progressive pruning schedule synthesis	2
4	Mask computation kernel (2:4 and 4:8)	1
5	CUTLASS 2:4 FP16 integration	2
6	CUTLASS 2:4 FP8 integration (CFTP composition)	2
7	CUTLASS NVFP4 4:8 integration (Blackwell)	2
8	WRGA sparse-base-with-dense-adapter kernel	2
9	Runtime pruning schedule executor	1
10	Correctness and benchmark tests	3
Total		18

Estimated scope: ~3100 LOC, 18 implementation sessions.

6. Discussion

6.1 Why this is the lowest-priority of the five gaps

Three reasons:

1. **Limited industry adoption.** Despite the hardware, 2:4 sparsity has not moved the needle in frontier production inference. The accuracy-recovery cost vs the kernel-speedup reality makes it marginal.
2. **Kernel maturity.** Sparse-kernel throughput is often $\sim 1.3\times$ in practice vs the theoretical $2\times$. The gap is closing as CUTLASS matures but is not closed.
3. **TDP-limited workloads.** Many NSL target workloads (training on single-node H100/Blackwell) are not primarily compute-bound; sparsity's compute savings don't translate to wall-clock savings proportionally.

That said, CSS is worth implementing for completeness (NSL should address every major technique domain) and for users specifically targeting Blackwell with 4:8 NVFP4 (where the hardware story is strongest).

6.2 When CSS becomes Tier 1

CSS becomes a higher priority if any of:

- Blackwell's 4:8 NVFP4 ecosystem matures (CUTLASS kernels, Triton support, etc.).
- A specific NSL use case emerges where sparse inference has strong economics (edge deployment, mobile).
- Accuracy recovery research produces better techniques (current SparseGPT/Wanda already achieve <1 point loss; below 0.1 would tip the balance).
- The outlier-safe pre-training line (OSP) combines with sparsity for training from scratch with good accuracy from day one.

6.3 Relation to other NSL papers

CSS composes with:

- **CFTP** (Innovation 5): sparse FP8 + 4-bit residual.
 - **CFIP** (Innovation 6): sparse + interior formats.
 - **WRGA** (Innovation 7): dense adapters on sparse base.
 - **CSHA** — attention weight matrices (W_q, W_k, W_v, W_o) are natural sparsification targets; the composition is straightforward (CSHA's attention kernel uses sparse weight MMAs).
 - **CTQS** — pruning schedule aligns with CTQS quantization phases; both are progressive, both have loss-guard semantics.
-

7. Conclusion

CSS provides NSL with first-class structured sparsity: 2:4 on Ampere/Hopper, 4:8 NVFP4 on Blackwell, with `Sparse<...>` as a working type, progressive pruning schedules synthesized at compile time, and compositions with CFTP, CFIP, WRGA, and CSHA. The technique is lower-priority than CCR, CKVQ, and CESH because industry adoption of structured sparsity is still catching up to hardware capability; but CSS is a natural completion of NSL's coverage of hardware-supported optimization techniques. When Blackwell's 4:8 NVFP4 ecosystem matures or when specific NSL use cases tip the economics toward sparsity, CSS will be ready.

References

1. NVIDIA. (2021). *Accelerating Sparse Deep Neural Networks*. Whitepaper on 2:4 structured sparsity.
2. Pool, J. et al. (2021). *Channel Permutations for N:M Sparsity*. NeurIPS 2021.
3. Frantar, E., Alistarh, D. (2023). *SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot*. ICML 2023.
4. Sun, M. et al. (2023). *A Simple and Effective Pruning Approach for Large Language Models*. arXiv:2306.11695. (Wanda.)
5. Red Hat Neural Magic. (2024). *2:4 Sparse Llama FP8: SOTA Performance for NVIDIA Hopper GPUs*.
6. NVIDIA. (2024). *Blackwell Architecture Whitepaper: 4:8 Structured Sparsity in NVFP4*.
7. PyTorch Team. (2024). *Accelerating Neural Network Training with Semi-Structured (2:4) Sparsity*.
8. Haziza, D. et al. (2025). *Accelerating Transformer Inference and Training with 2:4 Activation Sparsity*. arXiv:2503.16672.
9. NVIDIA. (2020). *Exploiting NVIDIA Ampere Structured Sparsity with cuSPARSELt*. Technical Blog.
10. HPC-AI.com. (2025). *Explore 2:4 Semi-Structured Sparsity with 1.27x Inference Speedup on NVIDIA GPUs*.